



哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

規格嚴格 功夫到家
1920 — 2017

Chapter 7: Collections and Strategy, Iterator Patterns



Collections and Strategy, Iterator Patterns

- **Overview of collection classes**
- **List interface and its standard implementation**
- **ArrayList and LinkedList**
- **Set interfaces with Map**
- **Strategy pattern**
- **Iterator pattern**

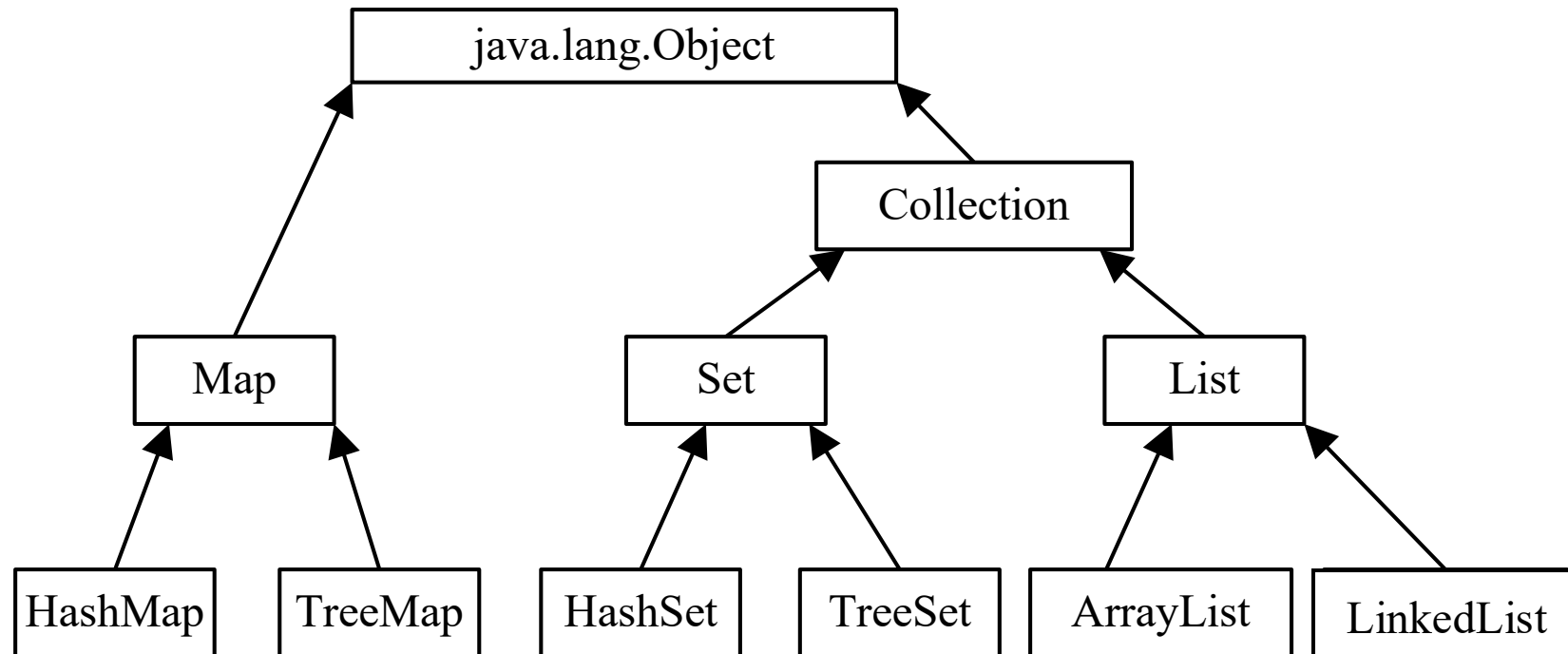


What is a collection?

- The Java language `java.util` package provides collection classes, which are also called containers. When it comes to containers, it is inevitable to think of arrays, and the difference between set classes and arrays is that **the length of the array is fixed and the length of the collection is variable.**
- The interfaces in the collection determine the basic characteristics of each class in the collection API. Specific classes simply provide different implementations of standard interfaces.

What is a collection?

- The most basic interface is the Collection interface, which defines the basic methods for manipulating data.
- Commonly used collections include List collection, Set collection, and Map collection, where List and Set implement the Collection interface. Each interface also provides different implementation classes.





List interface

- The List interface defines an **ordered** collection of objects that allow **duplicate** elements to exist.
- Lists are similar to dynamic arrays or variable-length arrays, and the number of elements stored in the List (the capacity of the List) can be automatically adjusted with insertion operations.



ArrayList class

- The ArrayList class is an array that can be dynamically modified, and the difference from ordinary arrays is that it has no fixed size limit, and we can add or remove elements.
- ArrayList inherits from AbstractList and implements the List interface.



ArrayList class

The ArrayList class is located in the java.util package, and it needs to be imported before use, and the syntax format is as follows:

```
import java.util.ArrayList;  
ArrayList<E> objectName = new ArrayList<E>();
```

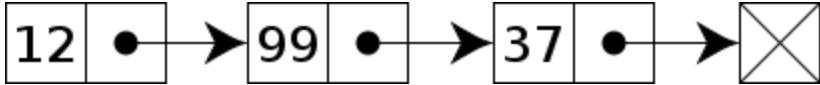
E: Generic data type, used to set the data type of objectName.

ArrayList is an array queue that provides related functions such as adding, deleting, modifying, traversing, etc.

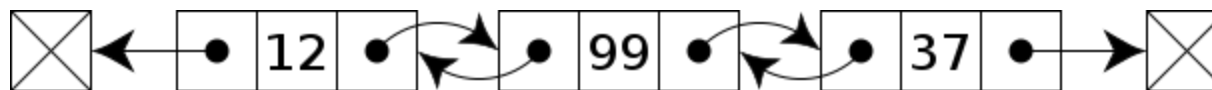


LinkedList class

- Linked list is a common underlying data structure, a linear table, but it does not store data in a linear order, but stores the address of the next node in each node. Linked lists can be divided into one-way linked lists and two-way linked lists.

- A one-way linked list contains two values: the value of the current node and a link to the next node. 

- A bidirectional linked list has three integer values: numeric, backward node link, forward node link.





LinkedList class

- Java LinkedList, similar to ArrayList, is a commonly used data container.
- Compared to ArrayList, LinkedList is more efficient for adding and deleting operations, while finding and modifying operations are less efficient. A one-way linked list contains two values: the value of the current node and a link to the next node.



LinkedList class

Note: Is it LinkedList faster than ArrayList when adding elements in the middle of a list?

Generally speaking, since the bottom layer of ArrayList is an array, adding data needs to move the data behind it, while LinkedList uses a linked list, just move the pointer directly, so LinkedList should be faster.

However, the selection of the insertion position has a great impact on LinkedList, because LinkedList needs to move the pointer to the specified node before it can start insertion, once the insertion position is farther away, LinkedList needs to move the pointer step by step until it moves to the insertion position, so the larger the insertion node value, the longer the LinkedList will take, because it takes time for the pointer to move. ArrayList is an array structure, and the position can be obtained directly according to the subscript, which saves the time of finding specific nodes, so the impact on ArrayList is not particularly large.

Summary: Although there are the above situations, because ArrayList can use subscripts to directly obtain data, ArrayList is generally selected when using queries, and LinkedList is more convenient when deleting and adding, so LinkedList is generally used more.



LinkedList class

Use ArrayList in the following cases:

- Frequent visits to an element in the list.
- Only add and remove elements are required at the end of the list.

Use LinkedList in the following cases:

- Frequent actions to add and remove elements at the beginning, middle, or specified location of the list are required.



LinkedList class

The LinkedList class is located in the java.util package, and it needs to be imported before use, and the syntax format is as follows:

```
import java.util.LinkedList;  
LinkedList<E> objectName = new LinkedList<E>(); // Constructor  
E: Generic data type, used to set the data type of objectName.
```



Set collection

- The objects in the Set collection are not sorted in a specific way, but simply add objects to the collection, but the Set collection **cannot contain duplicate objects**.
- The Set collection consists of the Set interface and the implementation class of the Set interface. The Set interface inherits the Collection interface and therefore contains all the methods of the Collection interface.



HashSet class

The HashSet class is located in the java.util package and needs to be introduced before using it, with the following syntax format:

```
import java.util.HashSet;
```

```
HashSet<E> sites = new HashSet<E>();
```

E: Generic data type, used to set objectName data type, can only be a reference data type.



HashSet class

Remove an element:

We can use the `remove()` method to remove elements from a collection:

```
sites.remove("Taobao"); // Delete an element, successful returns true, otherwise false
```

Deleting all elements in a collection can be done using the `clear` method:

```
sites.clear();
```



Map collection

- The Map interface provides objects that map key to value. A mapping cannot contain duplicate keys, each key can only be mapped to a maximum of one value. The Map interface also provides common methods for collections, as shown in the table below.

Method	Description
put(K key, V value)	Associates the specified value with the specified key. If the key already exists, its value is replaced.
get(Object key)	Returns the value to which the specified key is mapped, or null if the map contains no mapping for the key.
containsKey(Object key)	Returns true if this map contains a mapping for the specified key.
containsValue(Object value)	Returns true if this map maps one or more keys to the specified value.
keySet()	Returns a Set view of all the keys contained in this map.
values()	Returns a Collection view of all the values contained in this map.



HashMap

- A HashMap is a hash list that stores key-value mappings.
- HashMap implements the Map interface, which stores data according to the hashCode value of the key, has a fast access speed, allows up to one recorded key to be null.
- HashMaps are out of order, meaning they don't record the order in which they are inserted.



HashMap

The key and value types of HashMap can be the same or different, either key and value of the String type, or the key and value of the String type of the integer.

The HashMap class is located in the java.util package, and it needs to be introduced before use, and the syntax format is as follows:

```
import java.util.HashMap;
```

In the following example, we create a HashMap object Sites, an integer with a key and a value of type String:

```
HashMap<Integer, String> Sites = new HashMap<Integer, String>();
```



Which collection do you use?

You need to keep the order of students as they sign in to lab: “Alice”, “Bob”, “Alice”, “David”. Which collection do you use and what does it store after everyone signs in?

1. Use a List (for example, ArrayList).
2. Reason: List keeps insertion order and allows duplicates.
3. Stored result: [“Alice”, “Bob”, “Alice”, “David”]
4. You can access by index, like `list.get(0)` is “Alice”.



Which collection do you use?

You need to keep a list of unique course codes students are currently enrolled in: “CS101”, “CS101”, “MATH200”, “EE150”. Which collection do you use and what does it store?

1. Use a Set (for example, HashSet).
2. Reason: Set stores unique values only, no duplicates.
3. Stored result: [“CS101”, “MATH200”, “EE150”] (order is not guaranteed).
4. “CS101” only appears once even though it was added twice.



Which collection do you use?

You need to look up a student's GPA by their student ID:

ID 1001 → 3.8

ID 1002 → 3.5

ID 1003 → 3.9

Which collection do you use and how do you get the GPA for student 1002?

1. Use a Map (for example, HashMap).

2. Reason: Map links a key to a value. Key = student ID, Value = GPA.

3. Stored result:

1. 1001 → 3.8

2. 1002 → 3.5

3. 1003 → 3.9

4. To get the GPA for student 1002, call `map.get(1002)`, which returns 3.5₂₁



Strategy patterns

- Strategy Pattern

Using multiple if-else statements can make the code complex and difficult to maintain in situations where multiple algorithms are similar, while strategy patterns allow policies to change as objects change.

- Many times we can have multiple strategies to achieve our goals

- The way to travel, choose to ride a bicycle or take a car, each of which is a strategy
- ...



Strategy patterns



How do you separate the algorithm from the object so that the algorithm can change independently of the customer using it?

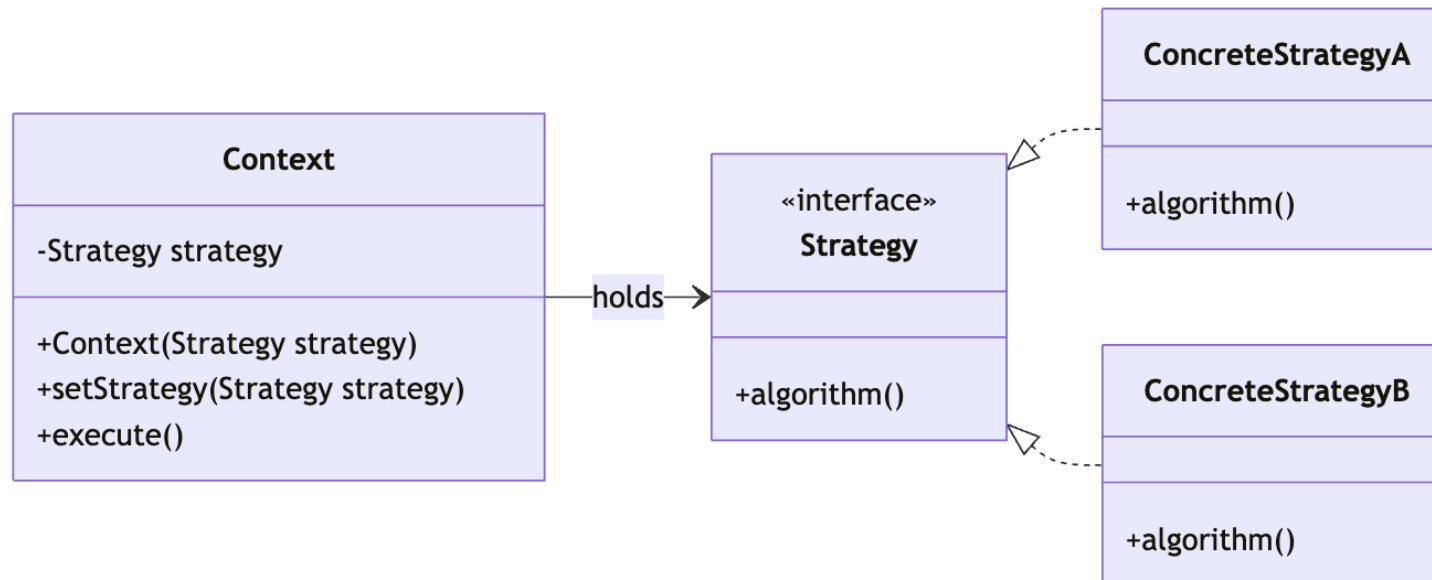
Define a series of algorithms, encapsulate each algorithm, and make them interchangeable with each other, so that the algorithm can change independently of the customer using it. And move the logical judgment to the client (i.e. let the client decide what specific policy to use in what situation).



Strategy Pattern

The strategy mode involves three roles:

- Abstract Strategy Role: This is an abstract role, usually implemented by an interface or abstract class. This role gives all the interfaces required for specific policy classes.
- ConcreteStrategy role: Wraps the relevant algorithm or behavior.
- Context role: Holds a reference to a Strategy.





Strategy Pattern

Review:

- As you can see from the example above, the policy pattern encapsulates the algorithm
- It is convenient to insert new algorithms into existing systems, and old algorithms to "retire" from the system to achieve method replacement
- It is up to the client to decide what algorithm to use in what situation.



Strategy Pattern

Note:

- The focus of the Strategy pattern: The focus of the policy pattern is not how to implement the algorithms, but how to organize and call these algorithms, so as to make the program structure more flexible, have better maintainability and scalability.
- Equality of algorithms: A major feature of the strategy model is the equality of each strategy algorithm. For a series of specific strategy algorithms, everyone's status is exactly the same, and it is precisely because of this equality that the algorithms can replace each other. All strategy algorithms are independent of each other in implementation and do not depend on each other.



Strategy Pattern

Note:

- Uniqueness of runtime policies: During operation, the pattern can only use one specific strategy implementation object at each moment, although it can dynamically switch between different strategy implementations, but only one can be used at the same time.
- Public behavior: It is often seen that all specific strategies have some public behavior. At this time, these public behaviors should be placed in the common abstract strategy role Strategy class. Of course, the abstract policy role must be implemented with Java abstract classes, not interfaces.



Strategy Pattern

Advantages :

- It provides an alternative to inheritance while maintaining the advantages of inheritance (code reuse) and being more flexible than inheritance (algorithmically independent and can be extended arbitrarily).
- It separates the logic of which algorithm to adopt or which behavior to take from the algorithm itself, avoids the use of multi-conditional transfer statements in the program, and makes the system more flexible and easy to scale.
- Adhere to most of the design principles, high cohesion, low coupling.



Strategy Pattern

Disadvantages :

- The client must know all the policy classes and decide for themselves which policy class to use. This means that the client must understand the difference between these algorithms in order to choose the appropriate algorithm class at the right time. In other words, the policy pattern is only applicable when the client knows the algorithm or behavior.
- Since the policy pattern encapsulates each specific policy implementation into a class, if there are many alternative strategies, the number of objects will be considerable.



Strategy Pattern

Summary of the strategy model:

element	description
Pattern Name	Strategy Pattern
Intent	Avoid the complexity and difficulty of maintaining multiple if-else statements when there are multiple algorithms that are similar.
Problem	How to separate the algorithm from the object so that the algorithm can change independently of the customer using it.
Solution	Define a series of algorithms, encapsulate each algorithm, and make them interchangeable with each other, so that the algorithm can change independently of the customer using it.
Consequence	Advantages: 1. The algorithm can be switched freely. 2. Avoid using multiple conditions to judge. 3. Good scalability. Disadvantages: 1. There will be more strategy classes. 2. All strategies need to be exposed.



Collections and Strategies, Iterator Patterns

- Overview of collection classes
- List interface and its standard implementation
- ArrayList and LinkedList
- Set interfaces with Map
- Strategy pattern
- **Iterator pattern**



Iterator Pattern

- **Iterator Pattern**

Inserting an iterator between the customer access class and the aggregate class separates the aggregate object from its traversal behavior, hides its internal details from the customer, and satisfies the "single responsibility principle" and "open and close principle".

- Iterator patterns are widely used in life

- The conveyor belt in the logistics system, no matter what items are transmitted, is packaged into boxes and has a unified QR code, which only needs to be checked one by one when distributing.
- When taking transportation, they all swipe their cards or face to enter the station, and they do not need to care about whether they are male or female, disabled or normal people.
- ...



Iterator Pattern

- A Java Iterator is not a collection, it is a method for accessing collections that can be used to iterate over collections such as `ArrayList` and `HashSet`.
- `Iterator` is the simplest implementation of Java iterators, and `ListIterator` is an interface in the Collection API that extends the `Iterator` interface.



Iterator Pattern

- The three basic operations of iterator are next, hasNext, and remove.
- Calling it.next() returns the next element of the iterator and updates the state of the iterator.
- Calling it.hasNext() is used to detect if there are still elements in the collection.
- Call it.remove() to remove the element returned by the iterator.
- The Iterator class is located in the java.util package and needs to be introduced before use, with the following syntax format:

```
import java.util.Iterator;
```



Thinking questions

□ How many for loops do you have?

```
for(int i = 0; i < myCollection.size(); i++) {  
    Item element = myCollection.get(i);  
    ...  
}
```

```
for(Item element : myCollection) { ... }
```

```
Iterator<Item> iterator = myCollection.iterator();  
while(iterator.hasNext()) {  
    Item element = iterator.next();  
    ...  
}
```

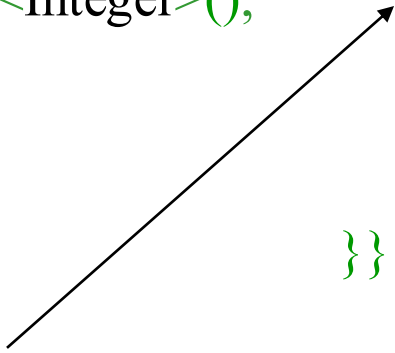


Iterator Pattern

Delete Element: To delete an element in a collection, you can use the `remove()` method. In the following example, we remove elements that are less than 10 in a collection:

```
import java.util.ArrayList;
import java.util.Iterator;
public class IteratorTest {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<Integer>();
        numbers.add(12);
        numbers.add(8);
        numbers.add(2);
        numbers.add(23);
        Iterator<Integer> it = numbers.iterator();

        while(it.hasNext()) {
            Integer i = it.next();
            if(i < 10) {
                // Delete elements that are less than 10
                it.remove();
            }
        }
        System.out.println(numbers);
    }
}
```



The output is: `[12, 23]`



Iterator Pattern

Iterator Pattern Summary:

element	description
Pattern Name	Iterator Pattern
Intent	Provides a way to access individual elements of an aggregate object sequentially without exposing the internal representation of the object.
Problem	1. Access the content of an aggregate object without exposing its internal representation. 2. It is necessary to provide multiple traversal methods for the aggregation object. 3. Provide a unified interface for traversing different aggregate structures.
Solution	Give the responsibility of moving between elements to the iterator, not the aggregate object.
Consequence	Pros: 1) It supports traversing an aggregate object in different ways. 2) Iterators simplify aggregation classes. 3) There can be multiple traversal on the same aggregate. 4) In iterator mode, it is convenient to add new aggregate classes and iterator classes without modifying the original code.



Iterator discussion

- An iterator is a pattern that separates the traversal behavior of a data structure of a sequence type from the object being traversed, i.e. we don't need to care about what the underlying structure of the sequence looks like
 - Case: If you use Iterator to traverse the elements in the collection, once you stop using List and use Set to organize the data, the code for traversing the elements does not need to be modified; If you use for to traverse, then all the algorithms that traverse this set will have to be adjusted accordingly, because the List is ordered, the Set is unordered, the structure is different, and their access algorithms are also different
- When looping, the Iterator can be used to delete the element; and cannot remove elements in a for loop



Comparison

▣ Advantages

- Supports multiple collection traversals
- The packaging is good, and the user only needs to get the iterator to iterate, and there is no need to worry about the traversal algorithm

▣ Disadvantages

- For simpler traversal (array or ordered list), using iterator traversal is cumbersome and inefficient
- Using iterators is more suitable for collections that are implemented in the form of linked lists at the bottom level